One Timeline, Many Renderings: A Wolfram Language Paclet for Heterogeneous Musical Output

AuthorA[1], AuthorB[2], and AuthorC[3] *

[1] InstituteA
[2] InstituteB
[3] InstituteC

Abstract. Algorithmic composition rarely ends in a single medium: the same musical plan may need to become a Csound score for synthesis, engraved notation for performers, a real-time control stream, and a click track for rehearsal. When each artifact is authored or repaired separately, their timelines drift apart. This paper presents Temporal System, a Wolfram Language paclet that treats heterogeneous output as a projection problem: one immutable store of typed musical entities, placed on a rational beat timeline, is compiled by per-backend renderers into Csound, MusicXML, and OSC artifacts. The paper describes the problem and the design decisions behind the paclet, its anatomy (temporal, semantic, and rendering layers across fourteen modules), and the implemented renderers. Particular attention goes to the Csound renderer, today the paclet's most complete backend: compiled tempo maps produce onset and duration seconds only at render time, session tuning produces p4 frequencies, and orchestras remain ordinary `.orc` files addressed by stable named instruments, continuing the score–orchestra separation that Csound inherits from the MUSIC N family. A dedicated click-track backend derives rehearsal audio from the same meter and tempo data and reuses the Csound serializer, showing how inexpensively a new projection can be added once the temporal core is shared. Figures include a timeline visualization exported directly from the paclet.

Keywords: Csound, algorithmic composition, music representation, Wolfram Language, click track

1 Introduction

Long before rendering became plural, computing composers had already separated structure from sound. At Bell Labs, James Tenney generated note lists for the MUSIC N programs to play [2,3]; in Utrecht, Gottfried Michael Koenig's *Project 1* and *Project 2* computed a piece as tables whose realization belonged to a later stage—players, the studio and, in his last electronic works, Csound itself [1]. In both practices the program owns *time and structure* while a separate system owns *sound*: the division Csound institutionalizes as the orchestra–score split [4].

Contemporary practice multiplies the realization stages. A mixed work may need a `.csd` for synthesis, MusicXML for engraving, a timed control stream for the live-electronics layer, and a click track for the performers. These formats encode time differently—seconds, divisions per quarter, milliseconds, audio samples—so maintaining them by hand means maintaining four diverging timelines. Computer-assisted composition environments have long addressed parts of this problem [5,6]; Temporal System, the Wolfram Language paclet described here, addresses it by making one authored timeline the only source of truth and demoting every output file to a derived artifact.

Its design decisions are few and strict: (i) events live in an immutable store of typed entities whose onsets are exact Rationals in beat space, never seconds; (ii) meter is authored state, never inferred from durations; (iii) reference tuning is a session parameter, never a constant in conversion code; (iv) the paclet generates *scores only*—synthesis instruments remain external `.orc` files, as in Koenig's and Tenney's workflows; (v) the shared store is permissive, while each renderer enforces a restrictive contract for its own payload.

* Please place acknowledgement here.

2 Anatomy of the Paclet

The paclet comprises fourteen modules exposing 79 public functions, organized in the three layers of Fig. 1: `core` (entities, store, tempo, session), `rendering` (dispatcher and four backends), and `util` (pitch, rhythm, dynamics, validation, timeline graphics). All functions are pure; store operations return new stores.

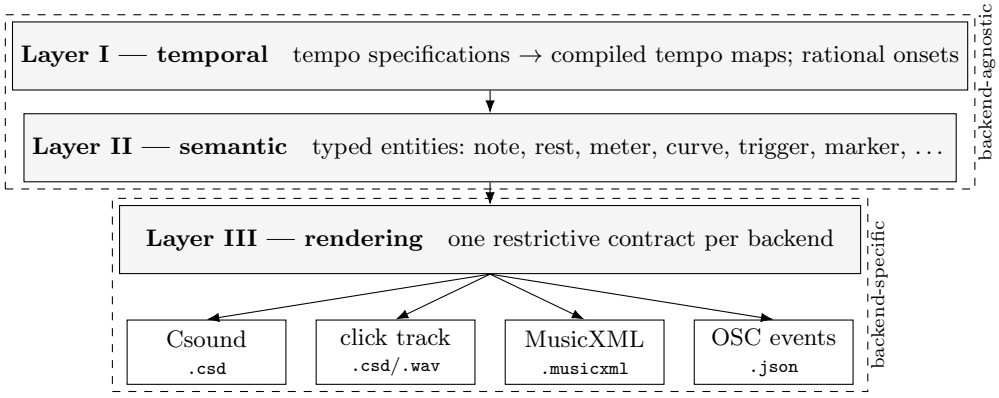

Fig. 1. The three-layer model. Adding a backend adds a contract in Layer III; Layers I and II never change to accommodate an output format.

Layer I. Tempo is authored as a *tempoSpec*, an ordered list of {beat, BPM} points, and compiled to a piecewise-linear *tempoMap* in {beat, seconds} form with exact analytical inversion. Onsets and durations are Rationals in musical duration space; conversion to seconds or samples happens only inside renderers.

Layer II. The store groups entities by type (`stemData[type][id]`, giving $O(1)$ typed access) across eleven types: note, rest, trigger, curve, process, state, marker, interval, trajectory, annotation, gesture. A note carries pitch (spelling plus cents, MIDI float, or frequency), duration, dynamic, and a `rendering` association keyed by backend id—the one place where backend-specific payloads travel with the entity. Meter is a state entity: measure structure is authored, not guessed. A session configuration carries everything a renderer may not hardcode, starting with `tuningA4` (default 440 Hz, freely reassignable).

Layer III. Constructors validate only the shared shape and require each `rendering[backend]` payload to be an association; the keys inside belong to the backend contract. Malformed Csound payloads are therefore Csound contract errors, local to one renderer, and each backend declares a fallback policy (warn, approximate) for material it cannot express.

The paclet also inspects itself: `drawTimeline` renders any store as lane-based graphics with the metric grid derived from the authored meter entities. Figure 2 is such an export—a segment of the harmonic series on one instrument, with an amplitude curve, a trigger, and a meter change—produced by a single function call, with no external tool.

3 The Implemented Renderers

A backend is data: an association with an id, a renderer, a serializer, the primitive types it emits, and its fallback policy. The generic dispatcher `renderTo[stemData, tempo, backend, config, registry]` is the single entry point for all outputs. Four backends exist. *Csound* compiles entities to score statements (Sect. 4). *Click* derives a standalone rehearsal track from meter and tempo alone (Sect. 5). *MusicXML* builds measures from meter state, splits notes at barlines into tied fragments, and validates measure capacity, part coverage, and tie and chord integrity; it is explicitly a lossy projection with a declared coverage profile [7], currently in beta and visually validated against a commercial engraving application. *OSC* emits a schedule-ready JSON event list in milliseconds—curves sampled at a configurable control rate—for dispatch as OSC messages [8] to whatever is listening: a graphical patcher as readily as a running Csound instance through its OSC opcodes. Not every entity

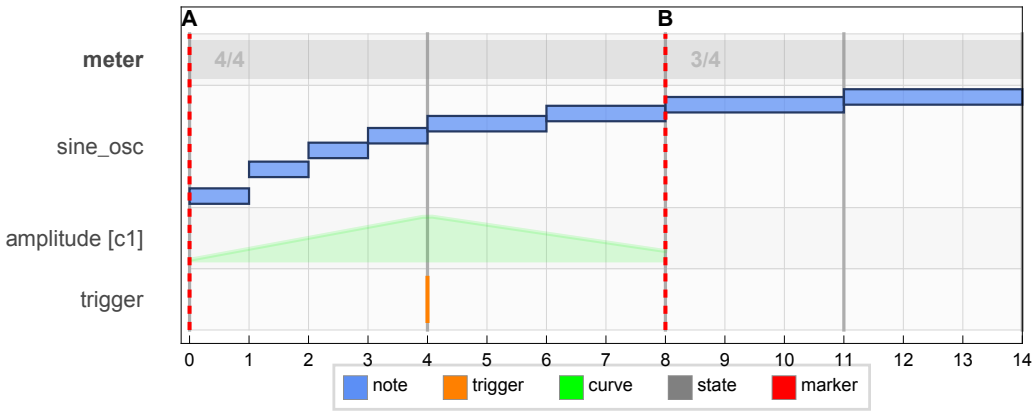

Fig. 2. A timeline exported directly by the paclet (`drawTimeline`): harmonics 2–9 of A2 as notes with cent offsets on a `sine_osc` lane, an amplitude curve, a `reverb` trigger, section markers, and a 4/4 → 3/4 meter change that reshapes the barline grid.

exists everywhere: curves become Csound f-tables but have no notation image; spelling matters to MusicXML and not to synthesis. The paclet records these fidelity differences per entity–backend pair instead of pretending projections are lossless.

4 The Csound Renderer

Csound is the oldest and most complete backend, and the one that currently fixes the paclet's rendering idiom (Fig. 3). Its p-field convention is stable: `p1` instrument, `p2` onset seconds, `p3` duration seconds, `p4` frequency in Hz, `p5` amplitude in $[0, 1]$, `p6+` passed through from the entity's `rendering["csound"]` payload. Notes and triggers become `i` statements; curves and trajectories become GEN −2 `f` tables sampled at 512 points; markers and annotations are skipped as non-sounding.

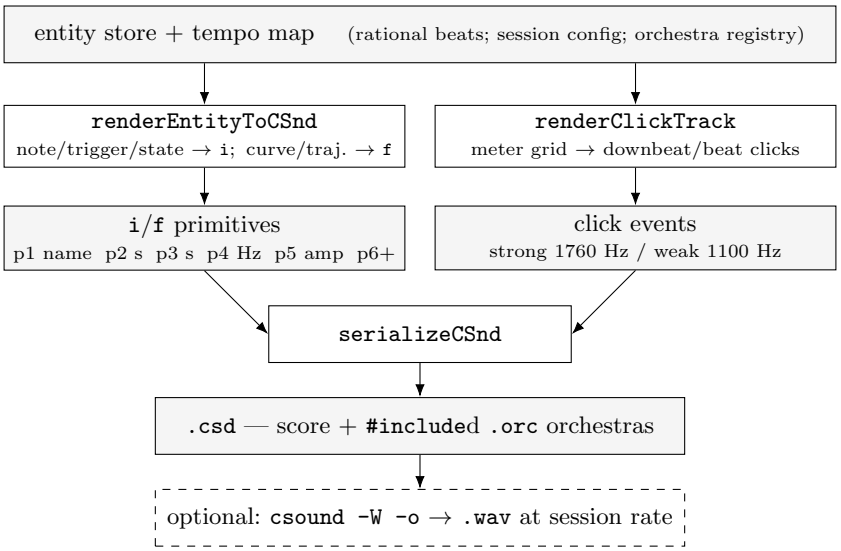

Fig. 3. Both Csound-family renderers emit the same primitives and share one serializer; instruments stay in external `.orc` files.

Two decisions matter most. First, *late conversion*: because onsets stay Rational until this point, `p2`/`p3` are computed from the compiled tempo map, `p4` from spelling and cents under the session's `tuningA4`, and `p5` from the dynamic marking, all at render time. Retuning a session to 432 Hz or editing its tempo specification regenerates every second and every frequency without touching one entity. Second, *named instruments*: an orchestra registry is loaded from a directory of `.orc` files, the filename being the stable id used both in `instr` definitions and in entity payloads. No numeric renumbering ever occurs, so the note

```
i "sine_osc" 0. 4. 440. 0.65 0.5
```

(A4, *mf*, two whole notes at 120 BPM, actual paclet output) remains readable and stable as the orchestra grows. The serializer sorts primitives by onset, writes `sr`, `ksmps`, `nchnls`, `0dbfs` from the session configuration, prefixes `#include` lines for the registry, and can optionally invoke the Csound binary to compile the exported `.csd` to audio—the test suite verifies the resulting sample rate.

5 A Click Track from the Same Timeline

Click tracks for mixed music are usually rebuilt by hand in a DAW whenever the score changes—precisely the divergence this architecture exists to remove. The click backend treats rehearsal audio as one more projection: it reads only the authored meter entities and the tempo map, builds the measure grid, and emits one short `i` event per beat, accenting each downbeat. Every parameter is session configuration—beat unit (1/4), click duration (1/16), strong/weak frequencies (1760/1100 Hz), amplitudes (0.9/0.45), and pan—and the click instrument itself is an ordinary three-line `.orc` file. Because the renderer walks the same meter grid and tempo map as every other backend, meter and tempo changes are followed with no additional authoring. The output below is actual paclet output for two 4/4 bars at 120 BPM (downbeats at 0 and 2 s):

```
<CsScore>
f 1 0 16384 10 1
i "click" 0. 0.125 1760. 0.9 0.5
i "click" 0.5 0.125 1100. 0.45 0.5
i "click" 1. 0.125 1100. 0.45 0.5
i "click" 1.5 0.125 1100. 0.45 0.5
i "click" 2. 0.125 1760. 0.9 0.5
...
```

The backend reuses `serializeCSnd` unchanged (Fig. 3), inheriting `#include` handling and optional compilation to `.wav` for distribution to performers. Implementing it required no change to Layers I–II: it is the working demonstration that a new projection costs one contract.

6 Conclusions

Temporal System contributes an architecture, not a synthesis method: an immutable rational-beat store from which Csound scores, click tracks, notation, and real-time event streams are compiled as unequal but coordinated projections. Csound currently plays the part of reference renderer because it makes the contract concrete—late conversion of time, pitch, and dynamics; stable named instruments; orchestras kept as Csound code. Current work includes stabilizing the MusicXML backend beyond beta and a hierarchical generation layer above the store. Sixty years after *Project 1* and the Bell Labs note-lists, separating computed time from rendered sound still pays: it is what lets one timeline become many renderings.

References

1. Koenig, G.M.: Project Two: A Programme for Musical Composition. Electronic Music Reports 3, 4–161. Institute of Sonology, Utrecht (1970)
2. Tenney, J.: Computer Music Experiences, 1961–1964. Electronic Music Reports 1(1), 23–60. Institute of Sonology, Utrecht (1969)
3. Mathews, M.V.: The Technology of Computer Music. MIT Press, Cambridge (1969)
4. Lazzarini, V., ffitch, J., Yi, S., Heintz, J., Brandtsegg, Ø., McCurdy, I.: Csound: A Sound and Music Computing System. Springer (2016)
5. Assayag, G., Rueda, C., Laurson, M., Agon, C., Delerue, O.: Computer-Assisted Composition at IRCAM: From PatchWork to OpenMusic. Computer Music Journal 23(3), 59–72 (1999)
6. Taube, H.: Notes from the Metalevel: Introduction to Algorithmic Music Composition. Taylor & Francis, London (2004)
7. Good, M.: MusicXML for Notation and Analysis. In: Hewlett, W.B., Selfridge-Field, E. (eds.) The Virtual Score: Representation, Retrieval, Restoration, pp. 113–124. MIT Press, Cambridge (2001)
8. Wright, M., Freed, A.: Open SoundControl: A New Protocol for Communicating with Sound Synthesizers. In: Proceedings of the International Computer Music Conference, pp. 101–104. Thessaloniki (1997)
9. Wolfram Research: Wolfram Language and Paclet Documentation. `https://reference.wolfram.com/language/`